

Implementation of the Dynamic Time Warping Algorithm for Bioacustic Detection

by

Elisabeth Christina Bosch

Dissertation presented for the degree of Electrical and Electronic Engineering in the Faculty of Engineering at Stellenbosch University

Supervisor: Prof. Jaco Versfeld

2026

Acknowledgements

* Supervisor X.
* Funding Y.

> *A very long quote*
> *from someone very important.*
> – Mr M

Thank you to Leanne for helping to prepare this template.

Declaration

By submitting this dissertation electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: day October 2026

Copyright © 2026 Stellenbosch University
All rights reserved.

Abstract

Table of Contents

Abstract **iii**

Table of contents **iv**

List of figures **vi**

List of tables **vi**

Nomenclature **viii**

Acronyms and abbreviations ix

Definitions x

1. Introduction **1**

1.1. Background 1

1.2. Problem Statement 1

1.3. Objectives 2

1.4. Scope 2

1.5. Layout of report 2

2. Literature **4**

2.1. Dynamic Time Warping 4

2.2. Feature Extraction Methods 7

2.2.1. Short Time Fourier Transform 7

2.2.2. Mel-Frequency Cepstral Coefficients 7

2.2.3. Linear Predictive Coding 9

2.3. Similar Studies 9

3. Dataset **10**

3.1. Background 10

3.2. Labeling 10

3.3. Data Splitting 10

3.3.1. Template Selection 11

3.3.2. Hyperparameter Tuning Template Selection 11

3.3.3. Evaluation Data Selection 12

4. System Design **13**
4.1. Overview 13
4.2. Feature Extraction Methods 13
4.2.1. STFT 13
4.2.2. MFCC 14
4.2.3. LPC 14
4.3. DTW methodology 14
4.4. Hyperparameter Tuning 15
4.5. Testing Procedure 18
4.5.1. Detection Methodology 18
4.5.2. Matching Methodology 20
4.6. Evaluation Metrics 21

5. Results **23**
5.1. Experiment 1: STFT Features 23
5.1.1. 5 Templates 23
5.1.2. 10 Templates 23
5.2. Experiment 2: MFCC Features 23
5.2.1. 5 Templates 23
5.2.1.1. No derivatives 23
5.2.1.2. First derivative 23
5.2.1.3. First and second derivatives 23
5.2.2. 10 Templates 23
5.2.2.1. No derivatives 23
5.2.2.2. First derivative 23
5.2.2.3. First and second derivatives 23
5.3. Experiment 3: LPC Features 23
5.3.1. 5 Templates 23
5.3.2. 10 Templates 23

6. Conclusion **24**

References **25**

A. Appendix title goes here **28**

List of Figures

2.1. An example of the optimal warping path between two vector features, X with length N = 4 and Y with length M = 6 4

2.2. An example for the three possible options for min as well as an example of the optimal warp path found for signals X and Y assuming X=[0, 2, 1, 0] and Y=[0, 0, 0.5, 2, 1, 0] 5

2.3. Example of a subsequence DTW alignment vetween two signal $X = [x_1, x_2, x_3, x_4]$ and $Y = [y_1.y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_10]$ 6

2.4. The mel scale filter bank is made up of 20 to 40 triangular filters arranged according to the mel scale. 8

4.1. This is a software flow diagram describing how the hyperparameters are chosen for the project. 15

4.2. This is a software flow diagram describing how the testing procedure runs for this project. 18

List of Tables

1.1. Your table caption 3

Nomenclature

Acronyms and Abbreviations

MFT My Funky Thesis

Definitions

S

Student is an entity needing a thesis to transcend the state of being a student.

Chapter 1

Introduction

1.1 Background

Since the late 1990s there have been indications that whales have been struck by shipping vessels.[1] Today this has become a larger concern, as vessel speeds and traffic has substantially increased. In False Bay, South Africa, a colony of Bryde's whales have been detected. Bryde's whales are found to migrate in smaller regions instead of the usual large-scale migrations of other whale species. Due to the increase in shipping traffic as well as the fishing activities in the area, the Bryde's whales in this are are also in danger.[2] Detection and classification systems for whale calls can be used to monitor the migration patterns of the specific whale colony throughout the year. The information gained from such analysis can help determine when vessel traffic should be diverted for the safety of the whale population. These types of systems have been implemented on different whale colonies, but the migration patterns of the False Bay colony has not yet been thoroughly documented.[3] Due to the variations in call types between different colonies of some whale species (killer whales[4], blue whales[5]), models trained for other Bryde's whale colonies would not be completely trustworthy for the detection of calls within the False Bay Bryde's Whale colony as universiality of calls cannot be assumed. Therefore, a model trained specifically for the False Bay colony would be beneficial. Dynamic Time Warping (DTW) has been shown to be an effective whale classification method on different whale colonies.[6]

1.2 Problem Statement

This project will determine whether using Dynamic Time Warping (DTW) in the frequency domain, thus using vector features, is able to effectively detect Bryde's Whale calls within the False Bay region. Three feature extraction methods will be compared. These will be

Short Time Fourier Transform (STFT), Mel-frequency Cepstral Coeffficients (MFCC) and Linear Predictive Coding (LPC).

1.3 Objectives

1. Implement a DTW detection system using three different feature extraction methods(STFT, MFCC and LPC). This shows whether the effectiveness of the DTW system is independent from the feature extraction method.
2. Calculate the precision, recall and F1-score for each model.
3. Compare results and determine whether DTW is overall a good detection method and which feature extraction method is best suited.

1.4 Scope

What you didn't consider.

1.5 Layout of report

* Chapter 2 provides anu background information needed to understand how to solve the given problem as well as any helpful literature of previous solutions to the problem.
* Chapter 3 provides an explanation of the overall setup and method used to solve the problem including what metrics were used to meet the stated objectives.
* Chapter 4 gives a detailed design explanation for all the functional blocks defined in the previous chapter.
* Chapter 5 provides evidence of results obtained after the system has been designed as well as descriptions of what the results show.
* Chapter 6 contains all concluding thoughts on the results and what that means for the solutions as well as improvements and recommendations for the project.

My funky thesis (MFT) is as good as MFT can be, as Table 1.1 and Figure **??** shows. Have a look at my explanation in Section **??** on page **??**, which is in Chapter 2.

Table 1.1: Your table caption

	Centered number [Units]	R-aligned number	Width-set col
Description	100	140	24601

Chapter 2

Literature

2.1 Dynamic Time Warping

DTW is an algorithm for detecting the similarity of two signals, $X = [x_1, x_2, \ldots, x_n]$ and $Y = [y_1, y_2, \ldots, y_m]$. This measure of similarity is achieved by calculating an optimal alignment cost between the two signals of length $N \in \mathbb{N}$ and $M \in \mathbb{N}$ respectively. In terms of feature vectors such as STFT or MFCC this means that each frame making up one signal is compared to each frame making up another signal. Using an alignment cost the best alignment for each frame in one to the other signal is chosen and an overall DTW cost is calculated to show how similar the overall signals are. Figure 2.1 shows that such an optimal alignment path may mean that a single frame in signal X may produce the lowest cost against more than one frame in signal Y.[7][8]

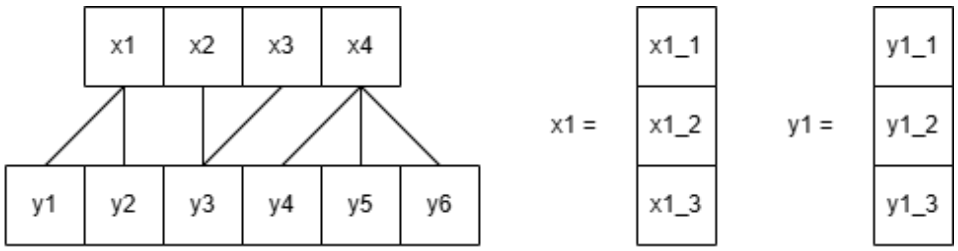


Figure 2.1: An example of the optimal warping path between two vector features, X with length N = 4 and Y with length M = 6

The overall cost needed to determine how similar the two signals are is calculated via a cost matrix. Cost matrix, $C_{N+1,M+1}$ for signals X and Y, is initialised such that $C_{i,0} = \infty$ for i = 1 to M, $C_{0,j} = \infty$ for j = 1 to M and $C_{0,0} = 0$.

The formula used to fill each cell of the cost matrix is:

$$C_{i,j} = \text{dist}(x_i, y_j) + \text{Q} \qquad \text{with} \quad \text{Q} = \min \begin{cases} C_{i-1,j-1} & \text{(match)} \\ C_{i-1,j} & \text{(insertion)} \\ C_{i,j-1} & \text{(deletion)} \end{cases} \tag{2.1}$$

$dist(x_i, y_j)$ represents the difference between X_i and Y_j as a distance. For vector signals this distance is calculated between each frame i of signal X and each fram j of signal Y. This can be done using various different techniques including cosine, euclidean or dot product similarity. For this project only the cosine similarity and euclidean distance metrics were used.[8]

Euclidean distance is sensitive to the magnitude and direction of the vectors for which it is calculated. In basic terms it is the absolute difference between two vectors.[9]

$$d(\mathbf{x_i}, \mathbf{y_j}) = \sqrt{(\mathbf{x_{i1}} - \mathbf{y_{j_1}})^2 + (\mathbf{x_{i2}} - \mathbf{y_{j_2}})^2 + \cdots + (\mathbf{x_{i_n}} - \mathbf{y_{j_n}})^2} \tag{2.2}$$

Cosine similarity is only sensitive to the direction of the vectors for which it is calculated. In basic terms it is the angle between the two vectors.[9]

$$\cos(\mathbf{x_i}, \mathbf{y_j}) = \frac{\mathbf{x_i} \cdot \mathbf{y_j}}{\|\mathbf{x_i}\| \, \|\mathbf{y_j}\|} \tag{2.3}$$

Q is the minimum value of three possible options namely match, insertion or deletion. Match is the value in the matrix cell diagonally down and left from the current cell for which the cost is being calculated. Insertion is the value in the matrix cell directly below the current cell and deletion is the matrix cell directly to the left of the current cell.

The cost which is then calculated for the current cell $C_{i,j}$ is a summation of the distance between the feature frames i and j in signals X and Y respectively as well as the lowest accumlated cost up until that point. This means that the final cell reached within the cost matrix, $C_{N+1,M+1}$ will contain the lowest overall cost for the similarity of signals X and Y. The lower the final cost, the higher the similarity between the two signals. The example shown in Figure 2.2 is shown if X and Y are chosen to be scalar signals, however the same logic is used for when X and Y are vectors.[8]

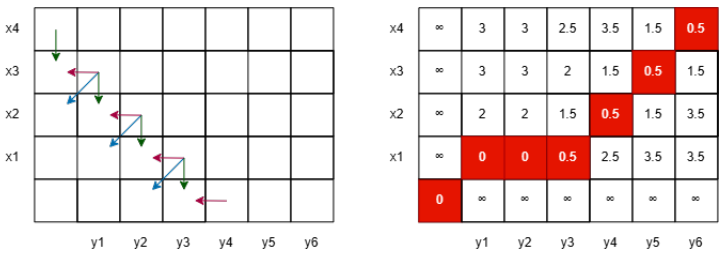


Figure 2.2: An example for the three possible options for min as well as an example of the optimal warp path found for signals X and Y assuming X=[0, 2, 1, 0] and Y=[0, 0, 0.5, 2, 1, 0]

The general case of DTW finds global alignment between the two signals. This alignment is found using the warping path. The warping path is the path of the best alignment between signals X and Y within the cost matrix, as shown in Figure 2.2. This path, W, is a sequence of matrix co-ordinates, (i, j), with a total length k.[10]

The boundary condition for global alignment ensures the final path starts and ends with the start and endframes of each respective signal.

$$i_1 = 1, i_k = N \quad \text{and} \quad j_1 = 1, j_k = M \tag{2.4}$$

The monotonicity condition ensures that each new point in the warp path is a step forward in time from the previous step.

$$i_{t-1} \leq i_t \quad \text{and} \quad j_{t-1} \leq j_t \tag{2.5}$$

The continuity condition ensures the step size of the warping path is only one unit in time.

$$i_t - i_{t-1} \leq 1 \quad \text{and} \quad j_t - j_{t-1} \leq 1 \tag{2.6}$$

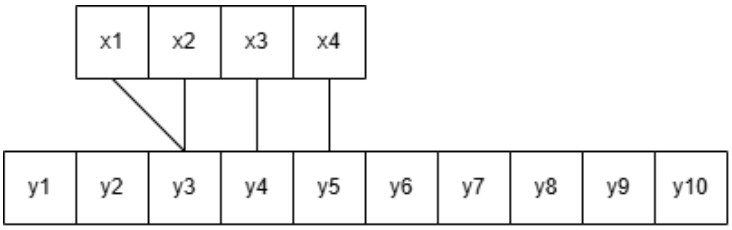


Figure 2.3: Example of a subsequence DTW alignment vetween two signal $X = [x_1, x_2, x_3, x_4]$ and $Y = [y_1.y_2, y_3, y_4, y_5, y_6, y_7, y_8, y_9, y_10]$

Subsequence DTW is a version of the DTW algorithm that alters the boundary condition such that global alignment is no longer necessary, instead the entirety of signal X is now matched with a subsequence of signal Y. The new boundary condition allows the warping path to start and end at anypoint within the first and final row of the matrix respectively.

$$i_1 = 1, i_k = N \quad \text{and} \quad 1 \leq j_1 \leq j_k \leq M \tag{2.7}$$

In global alignment the final cell in the cost matrix (N+1, M+1) is considered the overall optimal cost for those two signals. With subsequence DTW the entire final row of the matrix is checked to find the lowest cost. The position of that cost shows where the end point of the alignment detected via subsequence DTW is.[11]

2.2 Feature Extraction Methods

2.2.1 Short Time Fourier Transform

The STFT is used to analyse the change in frequency of a signal over time. The STFT of a segment of audio is calculated by multiplying the signal in the time domain, $x(t)$ with a window $w(t-\tau)$ which is sweeped across said signal. The Fourier Transform of each window is calculated and used to determine the frequency information for that window. Thus, the STFT provides the change in frequency for a segment of audio throughout time.[12][13] The mathematical formulation of STFT is:

$$X(\tau,\omega) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, e^{-j\omega t}\, dt, \tag{2.8}$$

where τ is the position of the window as it shifts through time at step-size H and ω is the frequency being analysed.

The frequency resolution of the STFT is given as:

$$resolution = \frac{f_s}{L} \tag{2.9}$$

where f_s is the sampling frequency of the signal and L is the length of the window overwhich the FT is taken.

This means that the STFT has a trade-off between time and frequency resolution.

If L is kept short, time resolution is high since smaller segments of time are being used for one FT analysis. However, frequency resolution is then decreased since too little time passes for the FT to distinguish between frequencies spaced closer together.

If L is kept long, time resolution drops. This means that fast changes in frequency will be missed. However, frequency resolution increases and it becomes easier to distinguish between differrent frequency bands.

Window overlap is another consideration when using STFT. Increased overlap creates a smoother representation of the changes in frequency occuring over time since information at the edges of the window is no longer lost. The trade-off with overlap is the increase in computational complexity that will occur due to smaller and thus more windows of a signal being analysed.[7][13]

2.2.2 Mel-Frequency Cepstral Coefficients

MFCC is a common choice for feature extraction when working on aucoustic data. This is because it is set up to percieve frequencies as they are percieved by the human ear. That means the lower frequencies are given a higher resolution than the high frequencies.[14] Features are initially extracted using the FFT. After this they are passed through a

Mel-scale Filter bank. The formula for the mel-frequency calculation is:

$$Mel(f) = 1127 \ln\left(1 + \frac{f}{700}\right) \tag{2.10}$$

with $Mel(f)$ as the frequency value on the mel scale and f as the actual frequency in Hz.[12]

This bank is made up 20 to 40 triangular filters which are placed ,smallest width to largest width according to the mel scale, along the frequency spectrum. The standard amount of triangular filters chosen is 26. This results in 26 coefficients for the entire range of frequencies being analysed.[15][14]

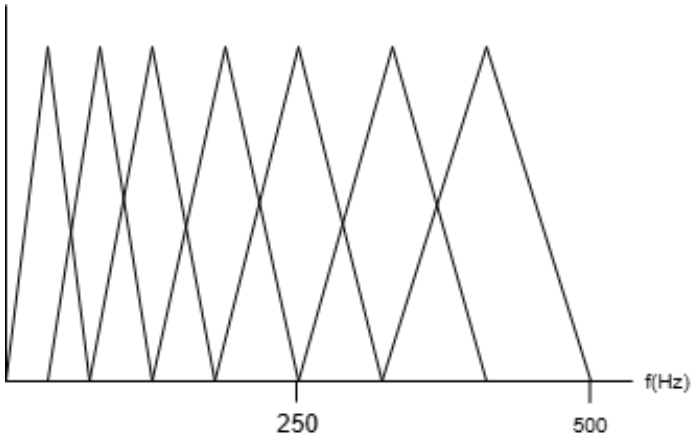


Figure 2.4: The mel scale filter bank is made up of 20 to 40 triangular filters arranged according to the mel scale.

The log of these coefficients is then taken and finally the Discrete cosine transform (DCT) is used to decorrelate the filter bank coefficients. The final number of cepstral coefficients returned by the DCT is typically 12 or 13 since the most important information is kept there. The formula use to perform the DCT is:

$$MFCC_i = \sum_{k=1}^{K} X_k \cos\left(\frac{i(k-0.5)\pi}{K}\right), \quad i = 1, 2, \ldots, K \tag{2.11}$$

with i as the index of the current coefficient, K as the total number of triangular filters used, k as the specific filter bank at that point in time and X_k as the energy after log was applied to the mel coefficients.[12]

The first and second derivatives, often referred to as deltas, of the cepstral coefficients are often used in feature extraction. They show the change in frequency between cepstral coefficients. This can be vital information when analysing how frequency changes across time since the MFCC process tends to lose this information. The formula for the derivatives is:

$$x_t^{(d)} = \frac{\sum_{m=-M}^{M} m \cdot x_{t+m}^{(d-1)}}{\sum_{m=-M}^{M} m^2} \tag{2.12}$$

with t as the specific point in time, d as the order of derivative and m as the frame offset.[16][17][18]

2.2.3 Linear Predictive Coding

2.3 Similar Studies

Chapter 3

Dataset

3.1 Background

The dataset used contains three 72 hour recordings, recorded in False Bay, near Gordons Bay, between 22 April 2023 and 22 June 2023. The recordings were taken at a depth of 20 to 30 meters at the coordinates, S 34.16813deg, E18.758718deg.

The device used for the recordings was a single omnidirectional hydrophone, designed and built by a Stellenbosch University research group from the Electrical and Electronic Engineering Department.

The recorded data has a sampling rate of 8kHz and a 16-bit resolution. The data was extracted as a 72 hour wav file from an SD card used by the recording device.[19]

3.2 Labeling

Detection tables, containg start and end times of calls within the each file, were provided from a previous research project which used a band-limited energy detector to flag calls. This method unfortunately did not detect all the calls within the audio files and none of the calls were labelled as a specific type of call.

Another detection table was procured. This table contained more detections for the first 72 hour file and contained call labels for all three files. This meant that there were sufficient labels for the first 72 hour recording of the dataset and there were partial labels for the second and third 72 hour recordings within the dataset.[19]

3.3 Data Splitting

For this project the data was plit in three sections. Each of the 72 hour audio files was chosen for a different section. Given that only the first of the three 72 hour audio files was

well labeled, it was decided that this would be used for training and testing. The second audio file was used for hyperparameter tuning and the third was used to select templates.

3.3.1 Template Selection

The audio file which was recorded starting on the 29th was used to select templates. These chosen templates were used for the comparison which determined whether an audio clip contained calls or not.

For the three different call type, single-tones, multi-tones and burst-tonals, the maximum number of templates per call type which could be extracted such that each call type had the same amount of templates was 10. The limiting call type was burst-tonals.

To ensure that the 10 templates chosen were representative of varying amounts of background noise templates were chosen by looking at spectrograms of detected calls throughout the audio file. The visual inspection was also necessary since some of the calls labeled as single-tones looked more like multi-tones and some of the multi-tones could actually be classified as burst-tonals.

3.3.2 Hyperparameter Tuning Template Selection

The audio file which was recorded starting on the 26th was used to select calibration templates. These templates were used to determine below which threshold a segment of audio could be considered a call. For the calibration templates 10 templates were chosen for each call type. This was again limited by the burst-tonals maximum number of templates. In this file there were however only 9 labeled burst-tonals, but when selecting multi-tones it was found that a call originally labeled as a multi-tone could in fact be seen as a burst-tonal.

Besides these 30 templates, 10 for each call type, 100 background templates were also selected. Within each audio file most of the time is made up of background noise and only some parts actually contain calls. This was simulated by selecting many more background templates that call-type templates. Since there is no table labeling background noise background segments of varying sizes were extracted from the file and 100 templates were manually selected. The reason for the manual process links with the unreliability of the labelled data. The audio file used was only labeled by the band-limited energy detector and thus did not label all the calls within the file. This means that some of the samples extracted as background audio actually contained calls that had never been labeled. Manually selecting each template ensured that all the templates chosen were in fact background noise only.

3.3.3 Evaluation Data Selection

The entire first audio file which was recorded starting on the 22nd was used for the final evaluation of the whale call detection model. No templates were extracted, instead the entire file was sweeped to search for calls.

The 22nd file was the only file which contained tonal-downsweep calls. Due to the limited amount of these types of calls all the detected tonal-downsweeps within this file was ignored while testing since thresholds could not be trained for this call type.

Chapter 4

System Design

4.1 Overview

The project was coded in Python. The functionality of each section of the project was placed in relevant .py files which were imported to the relevant jupyter notebook which was used for that section.

4.2 Feature Extraction Methods

The raw audio recorded by the hydrophone was stored in .wav files with a sampling frequency of 8kHz. Before extracting feature vectors from audio segments the audio was first downsampled to 1kHz sampling rate. This helped remove unecessary frequency information as the Nyquist Frequncy ($f_s/2$) for a 1kHz sampling rate is 500 Hz.[20]

Most of the Bryde's Whale vocalizations occur below 200 Hz. Thus, this sampling rate also ensures no relevant information was lost before analysis.[2]

4.2.1 STFT

The signal.stft function from the scipy library was used to calculate the STFT magnitudes for use as feature vectors. Window length L, as referenced in Eq.(2.9), is calculated by multiplying f_s, the sampling frequency of the given audio segment, with the chosen STFT frame duration. No set frame duration was selected as the optimal frame duration was chosen during the calibration stage of this project.

The window ovelrap H, as referenced in Eq.(2.9), is calculed by multiplying L with the chosen overlap percentage. For this project the overlap percentage was chosen as 0,75. The high overlap percentage produced a smooth spectrogram when viewing the time–frequency representation of the given audio segment.A Hamming window was chosen to decrease any

spectral leakage caused by the finite length of the windows used to extract the feature vectors.[13][21]

4.2.2 MFCC

The feature.melspectrogram and feature.mfcc functions from the librosa library were used to calculate the cepstral coefficients for the MFCC feature extraction implemetnation. The window length for the Fourier Transform required is calculated by multiplying f_s with the chosen frame duration which is optimally selected during calibration. Unlike the STFT function librosa.feature.mfcc requires the hop length and not the ovrelap length. This is calculated by multiplying $L - H$. H is also calculated by choosing the overlap percentage to be 0,75.Just as in the STFT feature extraction implementation a Hamming Window is used to decrease spectral leakage.

The librosa function used for calculating MFCC automatically uses DCT type 2 and ortho normalization. These settings mean that the bottom row of the output matrix, c_0, represents the sum of the log-spectrum energy. It is common practice in speech analysis to remove coefficient 0, as it contains total energy and can dominate subsequent analysis. For this project, only coefficients 1 to 14 were kept. This ensured that the analysis considered only the overall shape of the log spectrum and not the additive energy. The number of n_{mels} was chosen as 26, which is common practice in audio processing, and 14 coefficients were calculated because 13 are typically used, with the lowest coefficient 0 removed.[22][23][24]

4.2.3 LPC

4.3 DTW methodology

The sequence.dtw function from the librosa library was used to calculated the Dynamic Time Warping cost matix between a template and an audio segment. The distance matrix needed as input for the function was calculated using the spacial.distance.cdist function from the scipy library.

The distance function was given the template feature and the audio segent feature extracted either by STFT or MFCC as well as the metric specified for each extractor.The two metrics considered were cosine similarity and euclidean distance. As shown in Eq.(2.3) and Eq.(2.2), cosine similarity calculates the distance in angle between two points and eiclidean distance calculates the acual distance between the two point. For the STFT feature extraction method cosine similarity was used rather than euclidean distance since cosine does not depend on magnitude an thus the distance between a template and a call further away vs closer together will only be judged on the angular difference and not the difference in magnitude. This ensures that louder and softer calls of the same type can be classified by spectral shape only and not loudness. For the MFCC feature extraction

implementation the energy created by the call due to how close or far away it is from the recording device is removed when c_0 was removed as explained in Section4.2.2. This means that euclidean distance could be used without concern of the effect of magnitude on the distance since only spectral shape is left to be analysed. Euclidean distance is the common choice for DTW analysis, therefore since the main cosine similarity benefit is not necessary for MFCC Euclidean distance was chosen.[25]

The subseq setting with in the sequence.dtw function was set to True. As explaines in Section2.1, subsequence DTW means that the start and end points of a detected match do not have to be the first and last points of both signals. When searching a 72 hour audio file for whale calls subsequence DTW allows for the detection of multiple calls within a single audio segment by searching the final row of the cost matrix for dips and choosing the lowest point of each dip as an end point of a call. This is further explained in Section**??**. [26]

4.4 Hyperparameter Tuning

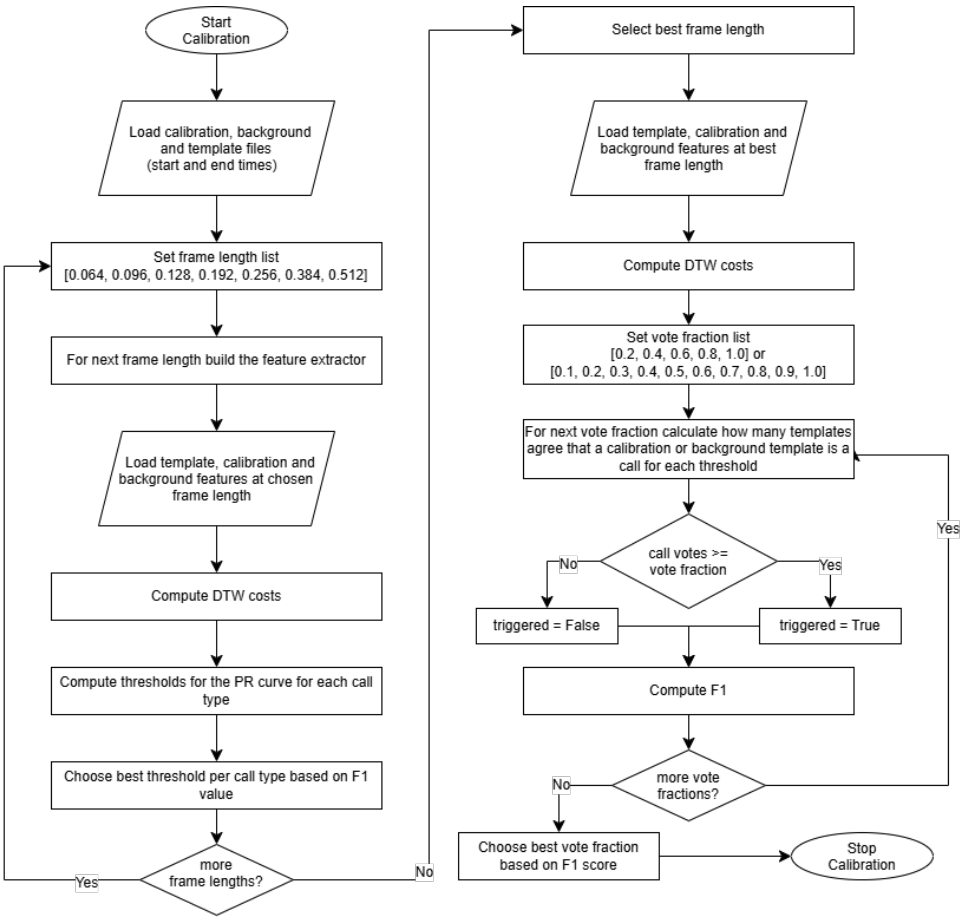


Figure 4.1: This is a software flow diagram describing how the hyperparameters are chosen for the project.

Three hyperparameters were chosen to be tuned, namely feature frame length, call type thresholds and call vote fraction. Figure4.1 show the software flow for the calibration part of the project where these hyperparameters are tuned.

As shown in Eq.2.9 frequency resolution= f_s/L. Considering the fact that Bryde's Whale single-tone calls tend to have a main frequency around 42Hz and multi-tone calls have more than one frequency between 42 and 100 Hz, upper bound for frequency resolution was chosen as about 10 Hz, this gives a frame length of 100ms. The lower bound in Hz was chosen as around 2Hz, this is due to the fact that while small frequency resolution is better for seperating frequency bins, 2Hz leaves a frame length of 500ms given a window overlap of 75% this leaves a hop length of 128ms. Given the average call length within the calibration file is 246ms this hop length is still short enough to detect the average call within such a file. Since it is computationally best to choose frame lengths wich are a power of 2 or multiples of powers of 2 the chosen frame lengths to be tested were (0.096, 0.128, 0.192, 0.384, 0.512).[27]

Since the calibration templates and background templates had to be handpicked due to data scarcity and since the subsequence DTW method was used each calibration template as well as each background template was padded. This padding allowed the subsequence DTW to work in conditions similar to those used in the final testing pipeline despit the data restrictions. It was ensure that the padded length was longer than the longest testing template being used with atleast one extra STFT frame length extra added to each side, this extra length was chosen to equal the longest feature frame length testes, 0.512, this ensures that there was always at least one frame length extra before and after the chosen segment. The call or background template was centered in this length. This was done by moving the start and ends times back and forward respectively to reach this set length.

For each of the tested frame lengths, three thresholds were determined, one for each call type. These thresholds were determined by taking the DTW costs calculated between testing and calibration templates for each call type as well as the DTW costs calculated between testing of each template and the background noise templates. These costs were stored in two seperate lists, one for calibration costs and one for background costs. The lowest and highest costs between the two lists was taken and 200 thresholds were selected between these two numbers in a linearly spaced fashion. The F1 values were calculated at each threshold for that call type as explained in Section4.6. The final threshold chosen for the specific call type was the threshold with the highest F1 score.

For each feature frame length there were therefore three thresholds chosen, one for each call type. The lowest F1 score out of these three thresholds F1 scores was saved. These saved F1 scores were then used such that the best feature frame length was chosen as the one where the lowest F1 score between the three thresholds for that frame length was the highest of all the lowest F1 scores.

While three thresholds were calculated, one for each call type, the project only focused

on general detection of calls and not on classification. This is due to the fact that many elements of each call type also appear in the other call type. Even though the three call types are different enough to justify three different thresholds and not one overarching threshold, calls are often misclassified. This choice is also due to the labeling discontinuity since different labelers had different opinions on what calssified a call as single-tine, multi-tone or burst tonal. This discontinuity means that the templates I selected for each call type, these were labelled by a different labeler than the person who labeled my testing data, might not match the manner in which another labeler labeled the testing data and thus my calibrated thresholds could differ to their labels.

For the main testing pipeling there are 5 or 10 templates against which each audio segment was tested. Since the templates were chosen to represent slight variations in call types this meant that while some templates would agree that an audio segment was or was not a call, not all would agree. A vote fraction was used such that if a certain percentage of the 10 templates agreed that an audio segment was a call then only would it be labeled as a call. A list of voting fractions were tested on the best frame length choice to determine the best vote fraction. When 5 templates were used the vote fractions tested were 0.2, 0.4, 0.6, 0.8 and 1.0. When 10 templates were used the vote fractions tested were 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9 and 1.0. The two options is due to the fact that 20% of 5 is 1 and 40% is 2 and so on, thus testing the odd numbered percentages would be unecessary.

For each vote fraction the F1 score for overall call detection was calculated. This means that each threshold was tested against all calibration templates no matter the call type as well as all background signals. This choice is due to the testing pipeline testing all audio segments against all three thresholds. The best vote fraction was chosen as the one with the highest overall F1 score.

4.5 Testing Procedure

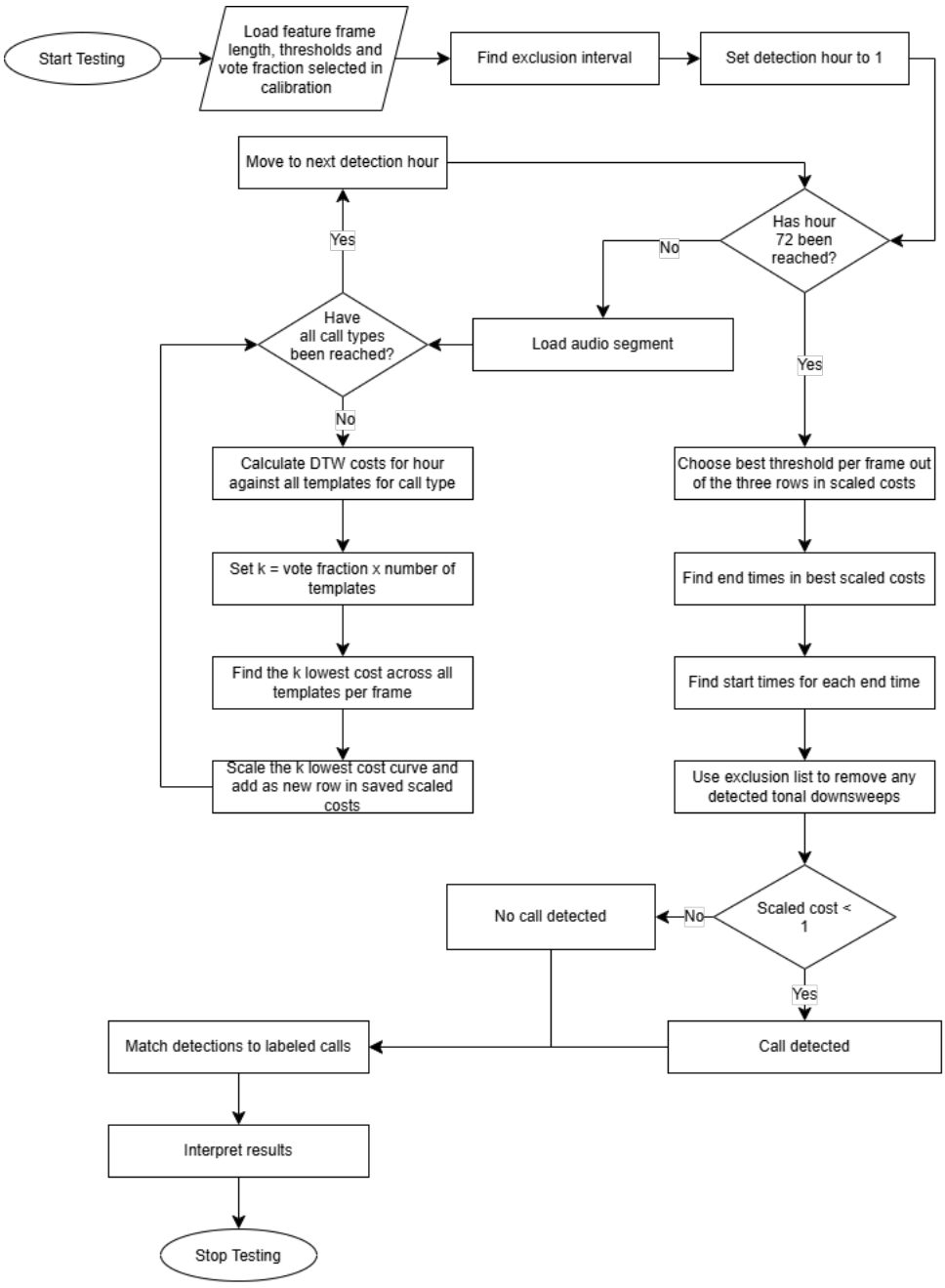


Figure 4.2: This is a software flow diagram describing how the testing procedure runs for this project.

4.5.1 Detection Methodology

The variables chosen during the calibration stage were used during testing. These were the feature frame length, thresholds per call type and the voting fraction. The file used for testing was the only file containing tonal downsweeps. Therefore, since no threshold

was calculated for this call type a list of exclusion indices was created for all the labled tonal downsweeps. Due to the use of subsequence DTW the tonal downsweeps were only removed after detecting calls since leaving gaps in time during subsequence DTW could effect the results.

The 72 hour file was processed one hour at a time. This ensured that memory usage was bounded to a smaller amount at one time. The soundfile library is used to load hourly segments of the 72 hour file without running through the entire file.[28] The extracted audio is then resampled from 8kHz to 1kHz using the resample_poly function from the SciPy library.[29] After this, features were extracted as explained in Section 4.2.1.

The DTW costs for all templates were calculated per call type and the bottom row of each DTW cost matrix for each template was stored in a matrix for every call type ($n_{\text{templates}} \times n_{\text{frames in the segment}}$). A variable k was calculated to be the vote fraction decided in calibration multiplied by the number of templates per call type. This variable was used such that the kth lowest cost across all templates per frame in the call type matrix was selected and stored in one kth lowest cost array. All the values in the array was divided by the threshold of the given call type from which the matrix came. This ensured that a resulting matrix of scaled kth costs per frame could be created such that no matter which call type the kth cost had been calculated from a cost of 1 would always indicate a match with the threshold. This scaled matrix was of size ($n_{\text{thresholds}} \times n_{\text{frames}}$). From the scaled matrix containing three kth best costs per threshold the overall best cost per frame was selected by choosing the lowest stored cost per frame out of the three options per frame.

Only the kth lowest cost was stored and analysed for call detection. Since the reason for the vote fraction was that if the percentage of templates for a sepecific call type which detected a call waws equal to the vote percentage then only would a call be detected, only the k lowest cost had to be considered since anything lower than this cost would have a lower cost and thus if the kth cost detected a call all lower calls would also detect a call and the vote fraction would be met. If, however, the kth cost did not detect a call even if any of the lower cost templates detected a call the vote fraction would not be met, thus a call would not be detected.

The find_peaks function from the SciPy library was used to detect end times within the scaled kth cost 1D-array. DTW works in such a way that the end point of a matching sequence is the lowest cost within the last row of the cost matrix. This means that within a 1D-array of a 1 hour segment with many calls there will be various dips in cost which will reach an absolute minimum cost at the end point of a possible call. If the costs considered were to be multiplied by negative one these end points would be found and peaks instead of dips. Thus any place wher $cost >= -1$ would indicate a detected end point of a call.[30]

For any detected end frame, e, within the scaled kth cost array the start time is calculated. From the ten templates for the call type within which the template against which the audio segment calculated the best cost found at end frame, e, the template

with the lowest cost at end frame, e, is selected. A section of the 1 hour audio segment is extracted such that it ends at the end of the chosen end frame and starts ten seconds before the end frame starts. When matching a template to an audio segment using DTW subsequence the detected call can be longer than the original template. Since there is no indication for how long the detected call could be a max of ten seconds was chosen. This is arbitrarily long for a Bryde's Whale call, but as long as the whale call starts somewhere within those ten seconds the correct start time can be found. The reason for the ten seconds is so that the DTW of the entire 1 hour audio segment does not have to be recomputed to find the warp path used to select the start time. This saves computational time.

The sequence.dtw function from librosa is once again used to calculate the DTW cost matrix of this audio segment. This function has the capability to return the best warp path for the given audio segment, however since the end point was selected on the k lowest cost template and the start time is now being chosenon the absolute lowest cost template, this template might have an end point with a higher cost that the selected end point. This end point may just not have been detected due to the vote fraction. Therefore, the step choice matrix needed to find the warp path is returned instead of the best warp path for that audio segment. The sequence.dtw_backtracking function from the librosa library is then used to find the warping path. In this function it is possible to force the warp path calculation to start f rom the chosen end point instead of the best for that audio segment. From that chosen end point the function works backwards though the steps matrix until it finds the start point.[31]

Theoretically whether one finds the start point in the kth lowest template or the best shouldn't make too much of a difference, however in the end the kth template was only used to find then end points due to the vote fraction. After finding those end points finding the most accurate start time is important and since the best template has matched the best to that call it m akes sense to find the start time using the best template instead if the kth best template.

4.5.2 Matching Methodology

For each detected call only the relevant time periods labeled calls were checked to find overlap. The searchsorted function from the numpy library was used to find the labeled calls overlapping the detected calls. Four 1D-arrays were used: start times for detected calls, end times for detected calls, start times for labeled calls, end times for labeled calls. For each detected call serachsorted was used to find at which index, within the array containing labeled call end times, the detected start time would have to be inserted to keep the end times array sorted from smallest to largest. This index was chosen as lo. The searchsorted function was also used to determine at which index within the start times

array for the labeled calls the end time from the detected call would have to be inserted to keep the start times array sorted. This was chosen as hi. These low and hi variables ensured that only labeled calls which started before the detected call ends and ended after the detected call start were considered. If hi was larger than lo then any indices between these two times were labeled calls touching the detected call. The matching logic was set up in such a way that if one or more labeled calls touched the detected call the call was considered to be a true positive call. So aslong as hi was larger than lo that detected call was labelled as matched and the labeled call or calls were labeled as matched. If another call were to also touch an already matched labeled call it would still be matched as general detection was the main goal and not specific time matches.[32]

4.6 Evaluation Metrics

While ROC analysis is more common in detection models, due to the class imbalance between whale calls present vs background noise PR analysis was chosen as the better method for analysis. This is due to the fact that Roc curves tend to give a false impression as to how a model acts in the presence of class imbalance. PR curves on the other hand take class imbalance into account and thus give a more accurate reflection of how the model acts. Due to this precision, recall and F1 score was chosen as the method of analysis.The one downside which was considered when interpreting results was that due to the lack of data the calibration data used had a different imbalance to the testing data. This meant that it was expected that the resulting chosen PR point would be less optimal than the PR curve would allow.

Precision is a value showing how many of the detected whale calls actually touched a label. True positives (TP) are defined as a detected call which overlaps at least one labeled call. False positives (FP) are defined as detected calls which do not overlap with any labeled call. The equation to calculate precision is defined as:

$$presision = \frac{TP}{TP + FP} \tag{4.1}$$

Recall is a value showing how many of the labeled calls were detected by the model. In this cas true positived (TP) are defined as a labeled call which has been overlapped at least once by a detected call. False negatives (FN) are the labeled calls which are never overlapped by a detected call. The equations for recall is defined as:

$$recall = \frac{TP}{TP + FN} \tag{4.2}$$

It can be seen that for precision and recall TP has a slightly different definition. This is due to the fact that the model at times could incorrectly detected one labelled call as

two different calls and sometimes two labelled calls can be detected as one call. Since the goal of this project is detection of alls and not classification the analysis does not consider the detection of one labeled call as two etected calls to be incorrect, thus leading to one of the detected calls being labelled as false positive. Instead any detection overlapping any labeled call and vice versa is considered true positive.

The F1 score shows the optimal point on a PR curve. This is due to the fact that as P increses and R increases F1 will also increase. Therefore, For all choices in hyperparameter tuning the choice is based on the highest F1 score. The F1 score is also used in the testing file to determine the overall performance of models relative to each other. The equation for F1 is defined as:

$$F1 = \frac{2PR}{P + R} \tag{4.3}$$

[33][34]

Chapter 5

Results

5.1 Experiment 1: STFT Features

5.1.1 5 Templates

5.1.2 10 Templates

5.2 Experiment 2: MFCC Features

5.2.1 5 Templates

5.2.1.1 No derivatives

5.2.1.2 First derivative

5.2.1.3 First and second derivatives

5.2.2 10 Templates

5.2.2.1 No derivatives

5.2.2.2 First derivative

5.2.2.3 First and second derivatives

5.3 Experiment 3: LPC Features

5.3.1 5 Templates

5.3.2 10 Templates

Chapter 6

Conclusion

References

[1] K. van Waerebeek, A. N. Baker, F. Félix, J. Gedamke, M. Iñiguez, G. P. Sanino, E. Secchi, D. Sutaria, A. van Helden, and Y. Wang, "Vessel collisions with small cetaceans worldwide and with large whales in the southern hemisphere; building a standardized database," 2006, https://www.vliz.be/imisdocs/publications/243193.pdf.

[2] A. Faustmann, F. W. Shabangu, and J. D. J. J. Versfeld, "Potential vocalizations of bryde's whales in false bay, south africa," 2024, https://repository.up.ac.za/server/api/core/bitstreams/57ba6a31-7aca-4acb-b9b6-60884695c964/content.

[3] S. Betz, K. Bohnsack, A. R. Callahan, L. E. Campbell, S. E. Green, and K. M. Labrum, "Reducing the risk of vessel strikes to endangered whales in the santa barbara channel," 2011, https://nmssanctuaries.blob.core.windows.net/sanctuaries-prod/media/archive/protect/shipstrike/pdfs/bren.pdf.

[4] V. B. Deecke, J. K. B. Ford, and P. Spong, "Call diversity in the north pacific killer whale populations: implications for dialect evolution and population history," *Animal Behaviour*, 2000, https://www.sciencedirect.com/science/article/abs/pii/S0003347211005537.

[5] M. A. McDonald, S. L. Mesnick, and J. A. Hildebrand, "The geographic variation of blue whale song," 2003, https://www.cetus.ucsd.edu/docs/reports/McDonaldIWC-2003.pdf.

[6] J. C. Brown, A. Hodgins-Davis, and P. J. O. Miller, "Classification of vocalizations of killer whales using dynamic time warping," 2006, https://pubs.aip.org/asa/jasa/article/119/3/EL34/850621.

[7] M. Müller and F. Zalkow, "Fundamentals of music processing: Dynamic time warping (dtw)," 2015, https://www.audiolabs-erlangen.de/resources/MIR/FMP/C3/C3S2_DTWbasic.html.

[8] H. Kamper, "Dynamic time warping (dtw)," 2021, https://www.youtube.com/playlist?list=PLmZlBIcArwhMJoGk5zpiRlkaHUqy5dLzL.

[9] R. Schwaber-Cohen, "Vector similarity explained," 2023, https://www.pinecone.io/learn/vector-similarity/.

[10] E. Alizadeh, "An illustrative explanation to dynamic time warping," 2020, https://towardsdatascience.com/an-illustrative-introduction-to-dynamic-time-warping-36aa98513b98/.

[11] M. Müller and F. Zalkow, "Fundamentals of music processing: Subsequence dtw," 2015, https://www.audiolabs-erlangen.de/resources/MIR/FMP/C7/C7S2_SubsequenceDTW.html.

[12] D. J. J. V. Atinde M. Usman, Olayinka O. Ocundile, "Review of automatic detection and classification techniques for cetacean vocalization," 2020, https://scholar.sun.ac.za/server/api/core/bitstreams/6e4a861d-74eb-4d2d-b81b-353743cc06b8/content.

[13] H. Kamper, "Short-time fourier transforms (stft)," 2026, https://www.youtube.com/watch?v=tQeDi2qEW-s&list=PLmZlBIcArwhMrUZ_njMmlO_T8gAWHihNS&index=26.

[14] ——, "Introduction to speech features," 2021, https://www.youtube.com/playlist?list=PLmZlBIcArwhN8nFJ8VL1jLM2Qe7YCcmAb.

[15] T. Singh, "Mfcc's made easy," 2019, https://medium.com/@tanveer9812/mfccs-made-easy-7ef383006040.

[16] DataMListic, "Spectral features - deltas and delta-deltas explained," 2023, https://www.youtube.com/watch?v=zxEnuPolylY.

[17] ——, "Mel frequency cepstral coefficients (mfcc) explained," 2023, https://www.youtube.com/watch?v=SJo7vPgRlBQ.

[18] S. A. Majeed, H. Husain, S. A. Samad, and T. F. Idbeaa, "Mel frequency cepstral coefficients (mfcc) feature extraction enhancement in the application of speech recognition: A comparison study," 2015, https://www.jatit.org/volumes/Vol79No1/5Vol79No1.pdf.

[19] A. Faustmann, "Potential vocalizations of bryde's whales in false bay, south africa," 2024, https://scholardata.sun.ac.za/articles/dataset/An_Analysis_of_Potential_Low_Frequency_Vocalizations_of_Southern_African_Bryde_s_Whales_/25108016?file=44351747.

[20] B. Meddins, "The nyquist frequency," 2000, https://www.sciencedirect.com/topics/computer-science/nyquist-frequency.

[21] A. M. Learning, "Windowing functions explained," 2026, https://apxml.com/courses/introduction-to-speech-recognition/chapter-2-processing-audio-signals/windowing-functions-explained.

[22] T. S. community, "dct," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.fft.dct.html.

[23] M. A. Roch, A. Širović, and S. Baumann-Pickering, "Detection, classification, and localization of cetaceans by groups at the scripps institution of oceanography and san diego state university," 2013, https://www.cetus.ucsd.edu/docs/publications/RochDCL2013.pdf.

[24] librosa development team, "librosa.feature.mfcc," 2026, https://librosa.org/doc/main/api/generated/librosa.feature.mfcc.html.

[25] T. S. community, "scipy.spatian.distnace," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html.

[26] librosa development team, "librosa.sequence.dtw," 2026, https://librosa.org/doc/0.11.0/generated/librosa.sequence.dtw.html.

[27] P. Hees, "The fast fourier transform, how and why it works," 2026, https://research.ibm.com/blog/the-fast-fourier-transform-how-and-why-it-works.

[28] B. Bechtold, "soundfile 0.14.0," 2026, https://pypi.org/project/soundfile/.

[29] T. S. community, "resample_poly," 2008, http://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html.

[30] ——, "find_peaks," 2008, https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html.

[31] librosa development team, "Source code for librosa.sequence," 2026, https://librosa.org/doc/latest/_modules/librosa/sequence.html.

[32] N. Developers, "numpy.searchsorted," 2026, https://numpy.org/doc/stable/reference/generated/numpy.searchsorted.html.

[33] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets," 2015, https://www.researchgate.net/publication/273155496_The_Precision-Recall_Plot_Is_More_Informative_than_the_ROC_Plot_When_Evaluating_Binary_Classifiers_on_Imbalanced_Datasets.

[34] T. Fawcett, "An introduction to roc analysis," 2005, https://www.sciencedirect.com/science/article/pii/S016786550500303X.

Appendix A

Appendix title goes here